

Model Right-Sizing Internals

The short [Model Right-Sizing guide](#) ran one evaluator under a few model choices and kept the cheapest that still caught the bad tile. This notebook rebuilds that run and opens it up: the one task that runs under each model, the mechanical oracle that grades them, and the selector that reads every run and decides.

Setup

Load the launch helpers.

```
In [1]: import pathlib
import sys

def _find_visual_artifact_example_root():
    cwd = pathlib.Path.cwd().resolve()
    candidates = []
    for base in (cwd, *cwd.parents):
        candidates.append(base)
        candidates.append(base / "examples" / "notebooks" / "visual_artifact")
    for candidate in candidates:
        if (candidate / "shepherd_usecases" / "visual_artifact" / "launch.py").exists():
            return candidate
    raise RuntimeError(
        "Cannot find examples/notebooks/visual_artifact. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    )

example_root = _find_visual_artifact_example_root()
if str(example_root) not in sys.path:
    sys.path.insert(0, str(example_root))
try:
    from shepherd_usecases.visual_artifact import launch
    from shepherd_usecases.visual_artifact import viz
except Exception as exc:
    raise RuntimeError(
        "Could not import the visual-artifact notebook helpers. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    ) from exc

launch.bootstrap(example_root=example_root)
```

Build a run to inspect

Start by building the run this notebook takes apart. `launch.open_workspace` opens a workspace and one flow, holding the prompt and the fixed candidate tiles every model is graded on. Each run runs the same evaluator, changing one thing: the `model` it runs under. `launch.grade_runs` then scores the finished runs against a mechanical oracle. `runs` holds the completed evaluator runs and `graded` their oracle scores; the cells below read both.

```
In [2]: prompt = launch.default_prompt()
model_choices = launch.model_choices()
model_choices
```

```
Out[2]: {'high': 'opus', 'mid': 'sonnet', 'cheap': 'haiku'}
```

```
In [3]: workspace = launch.open_workspace("model-right-sizing-internals", prompt=proc
```

```
In [4]: runs = {}
for config_name, model in model_choices.items():
    runs[config_name] = launch.run_static(
        workspace,
        name=f"rightsize-{config_name}",
        output_path=launch.VERDICT_PATH,
        output_content=launch.evaluator_content(config_name, model),
        model=model,
        metadata={"model_tier": config_name, "model": model},
    )

graded = launch.grade_runs(runs)
```

The oracle every run is graded against

You could judge each run with a reviewer: a run that reads the output and forms an opinion. Right-sizing can't rely on that. The model is the thing on trial, so its judge has to be fixed and model-free. That judge is a mechanical oracle, a deterministic gate that already knows which candidate tile is broken. Grading a run is then a check for agreement: whether this model caught the failure the oracle knows about. Each `GradedRun` carries the model it ran under, its cost tier, whether it caught the hard failure, and whether it passed. Here is how the runs scored:

```
In [5]: viz.show(viz.table([
    {
        "config": name,
        "model": item.model,
        "cost": item.cost,
        "catches hard fail": item.catches_hard_fail,
        "state": "passed" if item.passed else "missed",
    }
    for name, item in graded.items()
]))
```

config	model	cost	catches hard fail	state
high	opus	high	True	passed
mid	sonnet	medium	True	passed
cheap	haiku	low	False	missed

The selector that decides

After every evaluator run finishes, the selector runs. It takes an artifact ref to each evaluator's `verdict.json`, follows those runs with `after`, grades them against that same oracle, and writes one `decision.json`: the cheapest model that still passed. Because it cites every verdict, the decision stays tied to exactly the runs it was based on.

```
In [6]: verdict_refs = {
    f"verdict_{name}": launch.artifact_ref(item.run, launch.VERDICT_PATH, la
    for name, item in graded.items()
}
selector = launch.run_with_artifact_refs(
    workspace,
    name="selector",
    refs=verdict_refs,
    output_path=launch.DECISION_PATH,
    output_content=launch.selector_content(graded),
    after=[item.run for item in graded.values()],
)
decision = launch.read_json(selector, launch.DECISION_PATH)
decision
```

```
Out[6]: {'dropped': ['high', 'cheap'], 'kept': 'mid', 'passed': ['high', 'mid']}
```

The durable run record

Each run leaves a record in the workspace: a durable `args_ref` naming the exact arguments it ran on, a digest of those arguments, and the input refs it cited. The selector cited every verdict, so its record carries them as input refs. `launch.run_record` and `launch.run_args` read that record back out of the workspace.

```
In [7]: record = launch.run_record(workspace, selector)
args = launch.run_args(workspace, selector)
{
    "run_ref": selector.run_ref,
    "args_ref": record.args_ref,
    "args_digest": record.args_digest,
    "input_ref_count": len(args["input_refs"]),
}
```

```
Out[7]: {'run_ref': 'run-5409ff7274d1',
        'args_ref': 'run-5409ff7274d1:args:32f168689bd01b91',
        'args_digest': 'sha256:32f168689bd01b91198ae6eb77c81b02bd28d3f917807891157672c71dcbaa4e',
        'input_ref_count': 3}
```

Flow trace

The selector joins the evaluator runs as one more run, with an edge from each evaluator into it, because it was handed those verdicts and read them.

`workspace.flow.trace()` regenerates that projection from the retained flow; the edges trace exactly what the decision was based on.

```
In [8]: viz.show(viz.flow_trace(workspace.flow.trace(), height="620px"))
```

Events

kind	run/flow	label	payload
flow.opened	flow-ad7ee9a16ff8		name='model-right-sizing-internals'
flow.fork.requested	run-f933248c3683		name='rightsize-high', after=[]
run.lifecycle	run-f933248c3683		task_id='shepherd_usecases.visual_a task_version='v1', status='retained'
provider.invocation	run-f933248c3683		execution_id='task-execution-b13052 invocation_id='static:8a457b6bff55' status='started', source='task_exec evidence_role='provider_provenance' model='opus', payload={'args_digest 'sha256:f003b0bc7aa15c51b4121a33502 'artifact_path': 'verdict.json', 't 'shepherd_usecases.visual_artifact.
provider.invocation	run-f933248c3683		execution_id='task-execution-b13052 invocation_id='static:8a457b6bff55' provider_event_kind='provider.invoc source='task_execution.metadata.pro event_id='static:8a457b6bff55:compl {'artifact_path': 'verdict.json', ' 'sha256:49363d1608afe149a67eb5cccd2 'launched_confined': False}
run.output.published	run-f933248c3683		output_name='workspace', output_id= output:cfcfc7ceb4f7f21c0f38b62ab08a binding='workspace', output_world_o
flow.fork.requested	run-e983183cd8c3		name='rightsize-mid', after=[]
run.lifecycle	run-e983183cd8c3		task_id='shepherd_usecases.visual_a task_version='v1', status='retained'
provider.invocation	run-e983183cd8c3		execution_id='task-execution-21d593 invocation_id='static:89cfb24f39f4' status='started', source='task_exec

The task that runs under each model

`tasks.static_artifact_task` is the single task the workspace ran under each model. Nothing about the task or its inputs changed between runs; only the `model` argument to `launch.run_static` did. This is what right-sizing does: hold the task and its inputs fixed, vary the model, and see which model still does the job. The contract below shows the task's inputs and docstring, without the provider-owned body.

```
In [9]: from shepherd_usecases.visual_artifact import tasks
viz.task_contract(tasks.static_artifact_task)
```

```
Out[9]: def static_artifact_task(
    repo: object,
    *,
    output_path: str,
    output_text: str | None = None,
    output_content: object | None = None,
    **artifact_refs: object,
) -> object:
    """Declare the static visual-artifact task executed by the provider runtime."""
    ...
```

Cleanup

All the runs — the evaluators and the selector — ran inside one workspace. Keep the decided tier as the selected output, discard the rest, and release the selector.

`workspace.close()` then releases the workspace. The decision is made, but the trace stays: every run, the model each used, and the verdicts the selector read to choose.

```
In [10]: for name, item in graded.items():
    if name == decision["kept"]:
        item.run.output().select()
    else:
        item.run.output().discard()
selector.output().release()
workspace.close()
```